读完这篇，你就搞懂 DeepSeek v4 了

原创 腾讯程序员 腾讯技术工程 2026年4月28日 02:23 广东 181人



作者：dorian

丨导语 2026 年 4 月 24 日上午，DeepSeek 又一次把"开源炸弹"丢进了大模型圈。没有预热，官微只有一句话："今天，我们全新系列模型 DeepSeek-V4 的预览版本正式上线并同步开源"。从评分上看，这次的模型已经非常接近"闭源三巨头"的水平了，同时也是当之无愧的"地表最强开源模型"。但细读这份技术报告「DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence」，会发现DeepSeek的工作远比评分更硬核，无论是架构创新还是工程优化都是一如既往的精雕细琢。

DeepSeek V4到底强在哪?

先看纸面参数:

DeepSeek-V4-Pro：1.6T参数，稀疏激活49B，1M上下文

DeepSeek-V4-Flash：284B参数，稀疏激活13B，1M上下文

有两个常见的小误解先破一下:

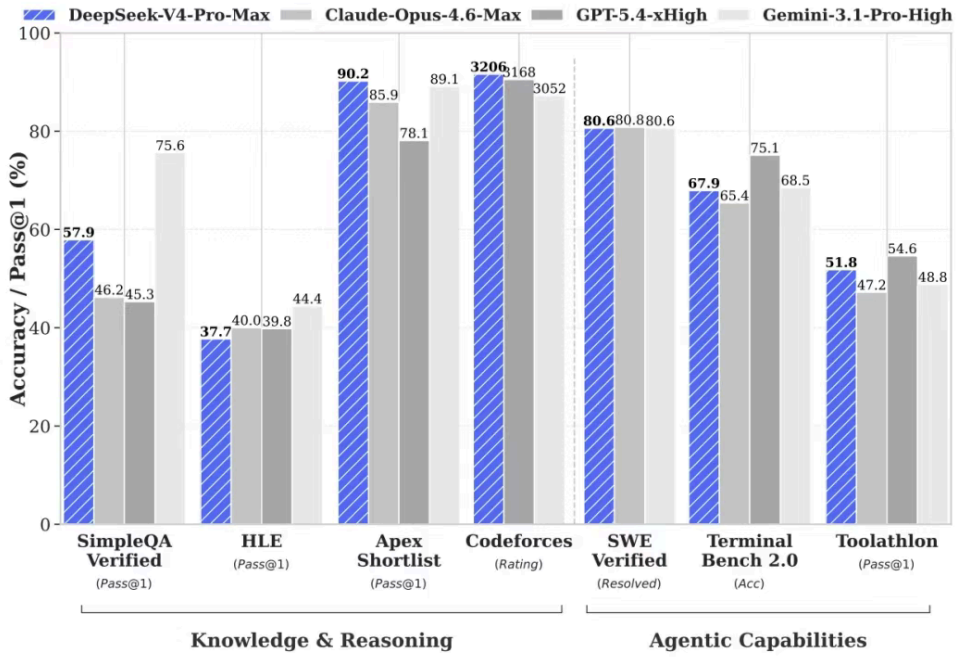
- **"Flash 是 Pro 的蒸馏小模型？"**

  —— 不是。两个都是独立预训练出来的 MoE，只是规模和稀疏度不同。

- **"1M 上下文是开关项？"**

  —— 不是。**两档都默认 1M 上下文**，服务端不再区分"长/短"模型。

再看评分：在编程、数学、Agent、长文本四个维度同时刷进第一梯队，基本接近GPT5.4、Claude 4.6及Gemini 3.1的水平，也是当之无愧的"开源最强"。



但V4真正硬核的地方绝不仅仅是1.6T参数+1M上下文，而是从 **attention 到 kernel 的系统级重构与优化**。DeepSeek团队这次到底做了什么工作，就让我们一起来深入浅出地解读一下。

开始之前的几个问题

为什么需要超长上下文?

或许在OpenClaw和Hermes这两个开源智能体框架走进公众视野之前，你可能会问"我平时用Chatbot也就几千token足够了，1M 上下文到底谁在用？是不是厂商又在军备竞赛？"，但当我们自己部署OpenClaw或Hermes成为自己的小助手时，发现还没问几个问题就已经消耗了几十万甚至上百万token了，同时随着智能体持久化记忆越来越多，传统的128k上下文窗口真的撑不住几轮问题，我来用3个典型场景来说明为什么会出现上述问题：

场景一：Agent 多轮任务轻松"吃"掉百万 token。

一个跑 30 轮的 Coding Agent，每轮往上下文里塞：用户指令（几百 token）+ 读了三四个源文件（几千到几万 token）+ 执行了若干 shell 命令和它们的 stdout（又是几千 token）+ 自己的 reasoning trace（几千 token）。**30 轮下来，数十万 token 起步是常态**。V3.2 为什么要在新用户消息来的时候丢掉 thinking history？不是它不想留，是**留不下**。

场景二：整仓库级代码理解 / 重构。

一个中型 Python 项目，300 个文件、15 万行代码，tokenize 之后大约 80–120 万 token。以前的做法是 RAG：向量检索挑几个相关文件塞进去。但**重构、跨文件一致性检查、类型推导**这些任务，RAG 漏一个调用点就是一个 bug。超长上下文的意义在于：**整仓扔进去，让模型自己看见所有调用关系**。V4 在 SWE Verified 上拿到 80.6 的成绩，和它能"一口气读完整个项目"直接相关。

场景三：长文档推理。

一份 200 页的法律合同、一份 500 页的学术综述、一套季度财报的四个附件——单个文档 50–80 万 token 很常见。以前切块摘要再合并，前后逻辑缺失巨大；现在直接让模型**在原始材料上做跨段落推理**。MRCR 1M 这个指标考的就是这个——V4-Pro 在这上面拿到 83.5，开源最高。

因此，1M 上下文绝不仅仅是"能写更长的 prompt"，而是让 Agent、整库代码、长文档等任务的执行真正可以高效执行并落地。

在 1M 上下文的时代，原来的 Transformer 为什么不够用了？

我们知道，在标准transformer架构模型的推理过程中：

- **Prefill 阶段**，整段上下文一口气进来，attention 的计算复杂度是O(L^2)——L提升 8×，prefill FLOPs 提升 64×
- **Decode 阶段**，每生成一个 token 都要把前面所有 L 组 KV 从 HBM 搬一遍，**显存占用和带宽压力都与 L 成正比**（MemKV∝L）

与此同时，任务越复杂、上下文越长，模型也要相应地**做得更深、更宽**，才能 hold 住跨段复杂推理和长程依赖——但深度和宽度一加，**层间标准残差的稳定性就成了新瓶颈**。

所以，传统 Transformer 架构要突破到 1M 上下文时代，必须同时解决三个问题：

1、如果标准残差机制中层间是一条统一固定的通道，那么需要升级为多条稳定可靠的通道，且每一层都可以通过'统一调度系统'来控制前序层的贡献值——**让更有价值的层获得更高的表达能力**。

2、GPU能在有效时间内算的过来，显存也能存的下。

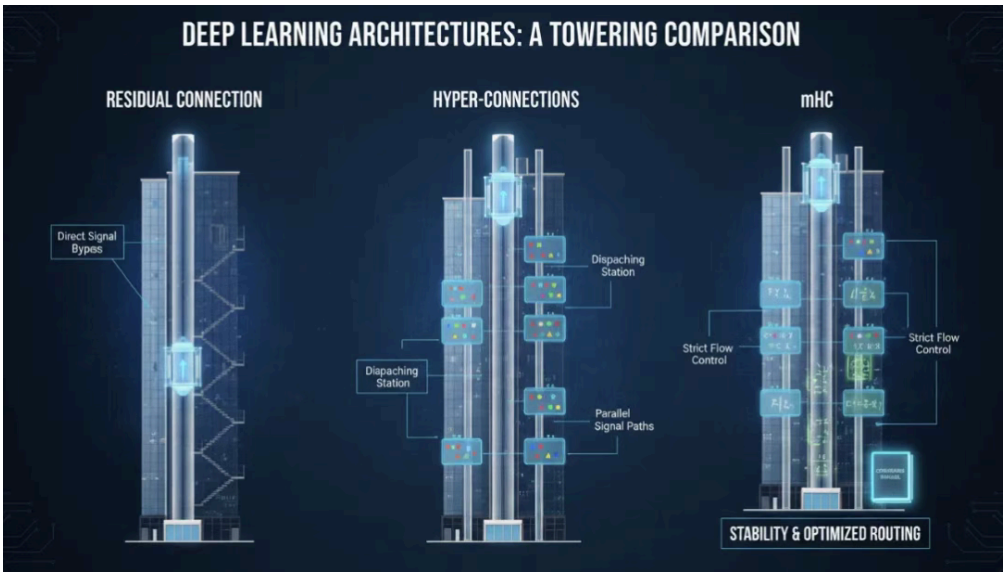
3、万亿参数、深度更大的模型网络训练更加稳定与规范化。

以上这三个问题，正是 DeepSeek V4 架构层面的三项核心创新分别给出的回应。

架构层面的创新机制

mHC：多流约束的残差连接

开始之前，请先看一下三种残差机制的形象比喻对比图，以方便理解：



1.标准残差：优雅但僵化

形象理解标准残差：深层网络就像一栋 100 层的大楼，每一层都有自己的工作要做。标准残差就好比在每一层旁边**修了一条通天的电梯井——**让底层的信号不用挤楼梯就能直接通往上面任意一层。

标准残差的严谨定义：

$$h_l = h_{l-1} + f_{l-1}(h_{l-1}) \quad (1)$$

也就是每一层的输入隐状态=前一层的输入隐状态+前一层变换（attn或ffn层）。我们将这个公式逐层展开，同时令h1=v0，v0就是原始embedding后的结果用于模型输入；vi代表每一层的变换结果f(hi)，得到如下公式：

$$h_l = \sum_{i=0}^{l-1} v_i \quad (2)$$

也就是说，每一层的输入，等于此前每一层变换结果的累加和，这也是标准残差的一条优雅性质。同时上述公式也说明了，历史每一层对于深层的贡献是均等的。但数学上的优雅，却带来了实践上的僵化，具体体现在以下三个方面：

容量瓶颈：所有层共用一条残差通路，浅层信息和深层信息都在同一条通路中"相互踩踏"，相互影响

路由僵硬：标准残差是"无论如何都要均等的接受每一层的贡献"，没有"阀门"可以调节"这一层我要哪些前序层分别多少贡献"，导致重要层的贡献被非重要层的贡献稀释

深度上限：万亿参数 + 百层以上时，由于多层累加的隐状态值无衰减累加的特性，导致标准残差会出现由**梯度消失 / 激活爆炸**带来的训练不稳定问题

为了解决以上问题，提出了改进的残差机制：多流残差机制Hyper-Connections。

2.标准Hyper-Connections

形象理解标准Hyper-Connections：我们还是看100层大楼的例子，如果说一栋 100 层的大楼，标准残差就好比在每一层旁边修了一条通天的电梯井，**但这栋楼里有人上楼，有人下楼，有人运货，大家挤在一起，会发生相互干扰。**Hyper-Connections的思路就可以比喻为以下两个思路：

1、既然一条通道相互干扰，那就多修几条电梯；

2、有人上楼，有人下楼，有人运货还是互相干扰，那我们就让乘客每一层都下电梯，大楼统一的调度系统重新按乘客目的调度一次，再重新上电梯（当然用电梯的例子比喻，这反而会降低效率，但从算法的角度上来看，出通道重新调度再进通道是很快的）。

标准Hyper-Connections严谨定义：

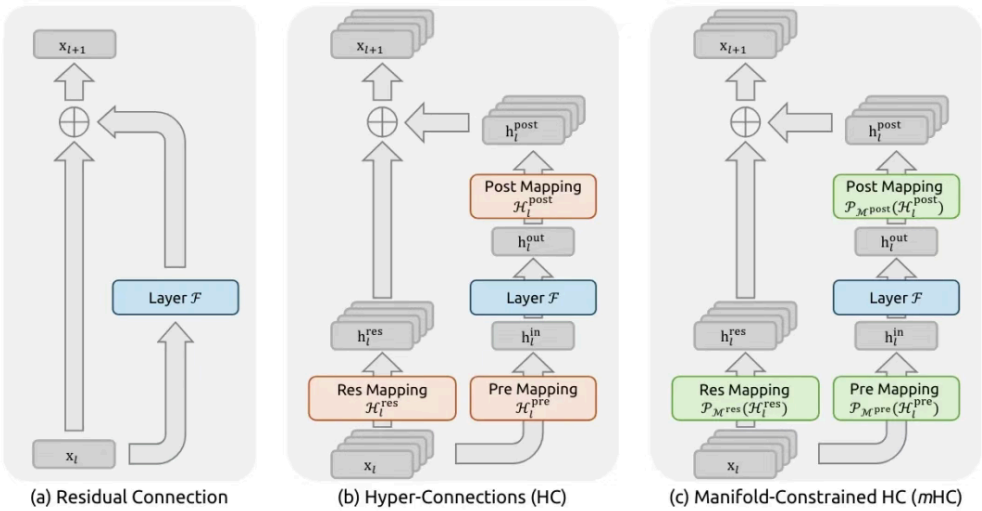
$$\mathbf{x}_{l+1} = \underbrace{\mathcal{H}_l^{\text{res}}\,\mathbf{x}_l}_{\substack{\text{残差调度} \\ n\times n \text{ 矩阵作用于 } n \text{ 条流}}} + \underbrace{\left(\mathcal{H}_l^{\text{post}}\right)^{\top}\mathcal{F}(\mathcal{H}_l^{\text{pre}}\,\mathbf{x}_l,\,\mathcal{W}_l)}_{\substack{\text{层输出按 } n \text{ 条流分发} \\ \text{先降流 } (n\to 1) \to \text{层变换 } \mathcal{F} \to \text{再升流 } (1\to n) \\ (1\times d) \to (n\times d)}} \quad (3)$$

层间残差连接维护了n个流，每次层间变换（ATTN or FFN）前，先进行降流为1，变换后的结构再升流为n，进行带权重的残差加和。同时把上述公式展开成公式(2)的形式，得到如下公式：

$$\mathbf{x}_L = \left(\prod_{i=1}^{L}\mathcal{H}_{L-i}^{\text{res}}\right)\mathbf{x}_0 + \sum_{i=0}^{L-1}\left(\prod_{j=1}^{L-1-i}\mathcal{H}_{L-j}^{\text{res}}\right)\mathcal{H}_i^{\text{post}\top}\mathcal{F}(\mathcal{H}_i^{\text{pre}}\mathbf{x}_i,\mathcal{W}_i), \quad (4)$$

可以清晰看到，这就是 HC 相对标准残差的本质区别：每层的贡献是带权重、多流并行的。

标准Hyper-Connections看似从理论上解决了标准残差的问题，但实际上每一层的系数是残差映射矩阵Hres的连乘计算，如果残差映射矩阵Hres、降流矩阵Hpre、升流矩阵Hpost都没有任何约束和限制的话，会导致隐状态值衰减/爆炸，直接导致**梯度消失 / 梯度爆炸**，为了给这三个矩阵加上“运营规范”，DeepSeek团队提出了mHC机制。



(a) Residual Connection (b) Hyper-Connections (HC) (c) Manifold-Constrained HC (*m*HC)

一张图看懂三种模式的演进：(a) 标准残差——**单流直传**；(b) HC——**多流 + 三个映射（pre/res/post）**，但映射完全自由；(c) mHC——**同样的多流 + 三个映射，但映射有约束保证稳定。**上文只讲到 (b)，(c) 留给下一节。

3.Manifold-Constrained Hyper-Connections：多流约束的残差连

形象理解mHC：如果说 HC 是给大楼修了多条电梯 + 配了统一的调度系统，那 mHC 就是**给调度系统装上了一部严谨的"运营规范"**，确保每次调度都不会让电梯超载、也不会让电梯空载。

mHC严谨定义：残差公式与标准HC的定义（公式3）完全一致，但是把残差映射矩阵Hres限制在一个特殊的集合 M 上（公式 5），而降流矩阵Hpre、升流矩阵Hpost仅需通过sigmoid限制在(0,1)即可

$$\mathcal{H}_l^{\text{res}} \in \mathcal{M} := \{M \in \mathbb{R}^{n\times n} \mid M\mathbf{1}_n = \mathbf{1}_n,\ \mathbf{1}_n^T M = \mathbf{1}_n^T,\ M \geqslant 0\} \qquad (5)$$

上述公式也称为「双随机矩阵」——每一行的和等于 1、每一列的和也等于 1、且所有元素非负，再加上sigmoid，这就是我们上面比喻的**运营规范**。

那么为什么需要把残差映射矩阵Hres限制为「双随机矩阵」，而降流矩阵Hpre、升流矩阵Hpost仅需限制在(0,1)即可呢？我们再回顾一下公式(4)，可以清晰发现：**历史每一层的系数都是残差映射矩阵Hres连乘的结果，而每一层变换前后使用的降流升流矩阵只和该层相关，**这就是mHC **"非对称约束"** 设计的精髓——**对高风险矩阵施加强约束，对低风险矩阵保持灵活。**而基于「双随机矩阵的乘法封闭特」，可以使连乘之后的结果矩阵依然是双随机矩阵，从而确保公式(4)中每一历史层的系数均保持在(0,1)之间，也就从根本上解决了梯度消失/梯度爆炸的问题。

另外，标准残差连接和它的变种（比如HighWay）实质上都是一种固定的函数表示路径，而mHC理论上可以达到通道数m种函数表示路径，这个结论也可以通过矩阵的半可分秩分析得到，也从数学上证明了“多个通道”提升了网络的表达能力。

4.其它残差优化机制

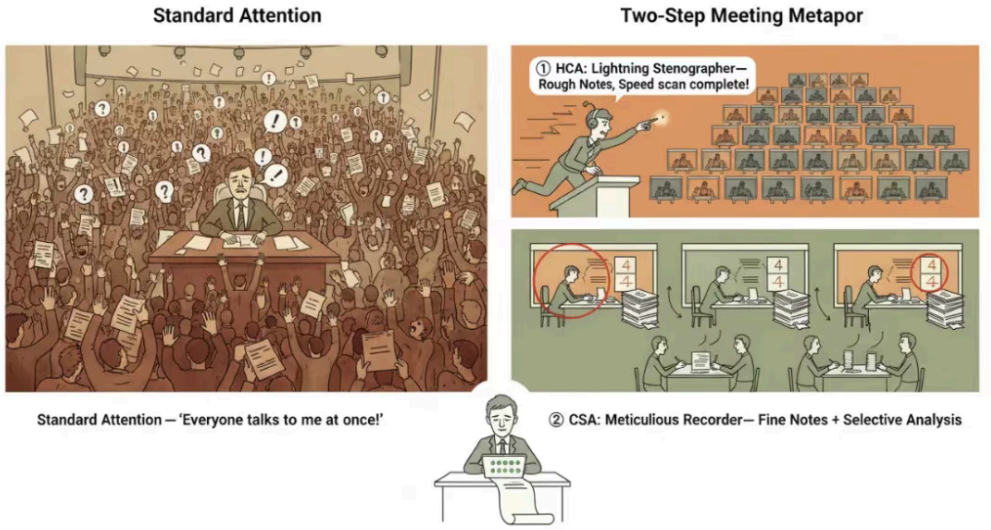
讲完了 DeepSeek 的 mHC，我们不妨把视野拉开——**看看业界在残差机制上还有哪些有意思的尝试。**这里简要介绍Kimi的注意力残差机制（Attn-Res）和分块注意力残差（Block Attn-Res），灵感同样是观察到了标准残差机制无权加和，层贡献逐层稀释的问题，直接将无权加和，改为有权加和，其中权重参数使用注意力机制计算得到。但在实践过程中发现，每层都对所有历史层做注意力回看开销极大，因此设计了折中方案分块注意力残差机制，即将transformer层进行分块，每个块聚合为单一表征，供后续阶段注意力查询使用，达到保证效果和开销之间权衡的目的。实践证明，块数设为8时，可以达到全注意力残差几乎同样的效果，同时将开销可控。

两种方案解决的是残差机制的不同侧面，没有绝对的优劣。这也是当前大模型架构设计最有意思的地方：同样的问题，不同团队给出了风格完全不同的解法。

CSA / HCA：混合稀疏注意力机制

标准 Transformer 里，每一层 attention 都要计算**序列里每一对 token 之间的关联。**在 4K 上下文的 ChatGPT 时代，这件事是**理论完美的，并且工程上跑得动的。**但把上下文窗口放大到 1M 的那一刻，**计算量立即膨胀6.5万倍，KV Cache立即膨胀256倍，既算不过来，显存也存不下**，这就意味着，**必须从"每一对都算"的范式里跳出来。**包含CSA和HCA两项核心创新的Hybrid Attention 就是 DeepSeek V4 给出的解法。

开始之前，还是先看形象比喻对比图，以方便理解：



1.Compressed Sparse Attention：压缩稀疏注意力

形象理解CSA：与标准注意力里 token 两两直接计算关联度的做法不同，**CSA 引入了一排"会议记录员"——**他们**不是会议参与者**，而是**站在会议外围、专门做纪要的第三方**。每位记录员负责把**相邻两组所有人的发言**综合成一份纪要。当要使用这些纪要时，通过索引找到高关联度的纪要，再逐一仔细查阅，最后得到自己所需的信息。我们把CSA形象比喻为三个步骤：

步骤一：每位记录员记录相邻两组的发言，也就是说，除了序列末尾的那一组，**每一组都会被两个记录员来记录**。而同一组的内容，自然也会被记录进两份不同的纪要。同时记录员并不机械地把每个人的话全抄下来——他会给每个人的每个"发言维度"独立打分：A 说的"时间维度"说得好，就多记录 A 的时间观点；B 说的"情感维度"说得深，就多记录 B 的情感观点。最终的纪要是两个组所有人在各自强项维度上的"最佳剪辑"——既压缩了信息，又保留了每个人的独到见解。

步骤二：当使用已经整理好的纪要时，我们通过索引的方式，找到与我们预期关联度高的这部分纪要。

步骤三：然后再逐一精细查阅这些高关联度的纪要，整理相关信息，最后得到结论。

我们就按着上述的三个步骤来逐一分析CSA的具体设计。

压缩机制（对应步骤一）：思路是把全部的token分组，每组m个token。在压缩的时候，首先要确保组内每个token是贡献加权的，其次通过把连续的两个组压缩内容拼接在一起来保证切割后语义的连贯性。

CSA压缩机制的数学表达式为公式(6)：

$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b \quad (6)$$

其中，公式左侧代表当前组的token的带权重加和压缩，公式右侧代表前一组token的带权重加和压缩（注意这里的计算是哈德玛积，不是矩阵乘法）。矩阵S为权重矩阵，定义为公式(7)：

$$[S^a_{mi:m(i+1)-1}; S^b_{m(i-1):mi-1}] = \text{Softmax}_{\text{row}}([Z^a_{mi:m(i+1)-1}+B^a; Z^b_{m(i-1):mi-1}+B^b]) \quad (7)$$

而计算上述内容需要的值投影矩阵C和矩阵Z由隐状态值投影得到：

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV},$$
$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ},$$

4个W矩阵和2个偏置矩阵B都是可学习的，在训练过程中得到。

闪电索引机制（对应步骤二）：思路与DeepSeek v3.2中的DSA机制一致，通过快速计算出每个token与压缩得到的全部“纪要”的关联度分数，同时每个注意力头加权后得到最终得分，随后选择关联度最高的k份纪要，供后续执行注意力计算时使用。

关联度分数计算方法为公式(8)：

$$I_{t,s} = \sum_{h=1}^{n_h^I} w_{t,h}^I \cdot \mathrm{ReLU}\left(\mathbf{q}_{t,h}^I \cdot K_s^{\mathrm{IComp}}\right) \quad (8)$$

其中K矩阵采用和公式(6)完全一致的计算思路，其实它也代表的是压缩结果的“key投影”。每个token的查询矩阵q和注意力每个头的权重矩阵w也均由隐状态值投影得到：

$$\mathbf{c}_t^Q = \mathbf{h}_t \cdot W^{DQ},$$
$$[\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \ldots; \mathbf{q}_{t,n_h^I}^I] = \mathbf{q}_t^I = \mathbf{c}_t^Q \cdot W^{IUQ},$$
$$[w_{t,1}^I; w_{t,2}^I; \ldots; w_{t,n_h^I}^I] = \mathbf{w}_t^I = \mathbf{h}_t \cdot W^w,$$

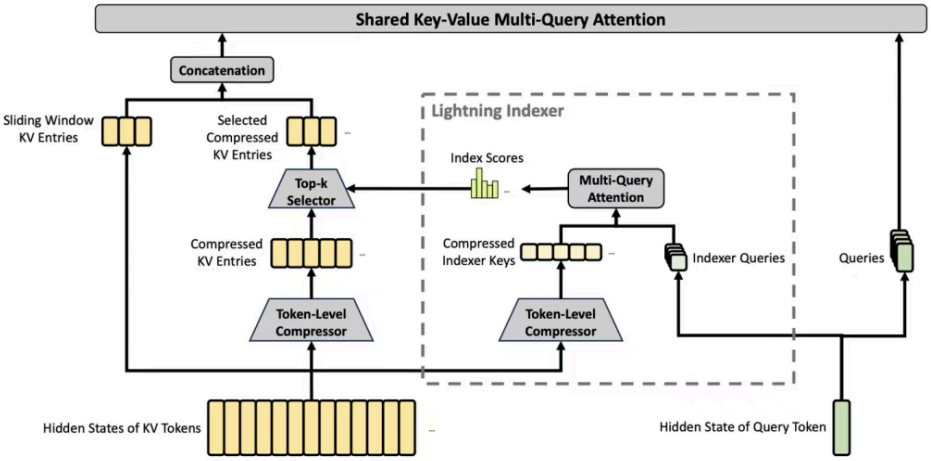
同样，投影计算用到的矩阵也都是通过训练得到的。

注意力计算（对应步骤三）：这里其实就是标准注意力计算了，只是共享了KV，注意力计算方法为公式(9)：

$$\mathbf{o}_{t,i} = \mathrm{CoreAttn}\left(\mathtt{query}{=}\mathbf{q}_{t,i},\ \mathtt{key}{=}C_t^{\mathrm{SprsComp}},\ \mathtt{value}{=}C_t^{\mathrm{SprsComp}}\right) \quad (9)$$

其中KV共享的C矩阵就是闪电索引得到的top-k采样得到的K组压缩过的信息，q是每个token的查询矩阵，计算方法不再赘述。

由此，则完成了token压缩+进一步稀疏采样的CSA机制，CSA完整的架构也可以参考下图：



2.Heavily Compressed Attention：高度压缩注意力

形象理解HCA：它像一个速览版的CSA，只是把“会议记录员”换成了“会议速记员”。在这个场景中，每场会议参与人数更多，“会议速记员”要记录内容更多的会议的纪要，但他仅需要聚焦于这场会议就可以了，而不需要同时关注“前后两场小会”。当需要使用这些会议纪要时，由于每份纪要都是多人数大会的纪要，因此总纪要份数并没有那么多，直接逐一查阅即可。我们把HCA形象比喻为两个步骤：

步骤一：每位速记员记录一场大会的全部发言，针对每个人发言的内容的处理逻辑同CSA中记录员一致。

步骤二：当使用这些纪要时，直接逐一查阅每份纪要，整理相关信息，最后得到结论。

同样，我们也按着上述的两个步骤来逐一分析HCA的具体设计。

压缩机制（对应步骤一）：与CSA的压缩思路类似，区别是每个组m'个token（m'≫m），但不需要连续两组压缩内容拼接在一起。

HCA压缩机制的数学表达式为公式(10)：

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j \quad (10)$$

可以清晰看到，HCA的压缩机制无需连续两组压缩内容的拼接，且加权计算、权重矩阵S和值投影矩阵C的计算方法与CSA中完全一致。

权重矩阵S定义为公式(11)：

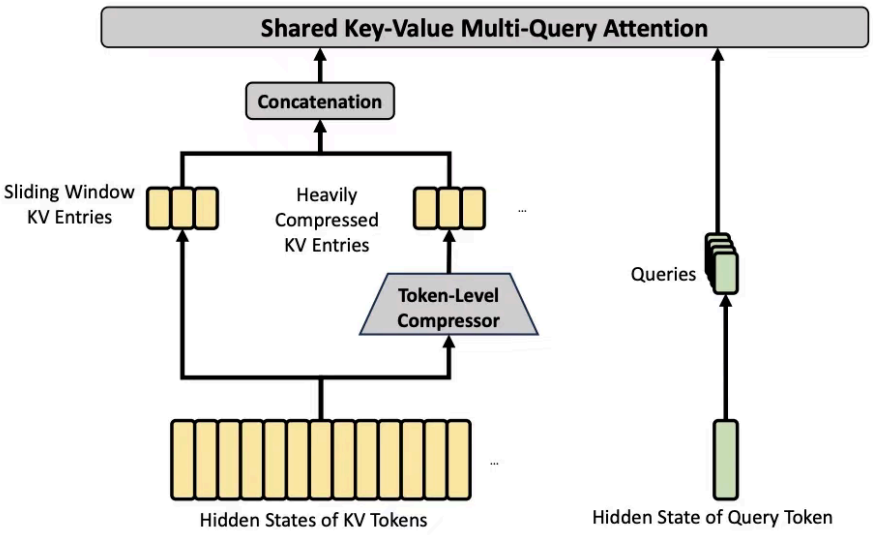
$$S_{m'i:m'(i+1)-1} = \text{Softmax}_{\text{row}}\left(Z_{m'i:m'(i+1)-1} + B\right) \quad (11)$$

值投影矩阵C和矩阵Z也同样由隐状态值投影得到，且2个W矩阵和偏置矩阵B同样也都是可学习的，在训练过程中得到：

$$C = H \cdot W^{KV},$$
$$Z = H \cdot W^{Z},$$

注意力计算（对应步骤二）：计算方法与CSA的注意力计算同样，公式同样参考公式(9)，区别是这里使用每个token的查询矩阵q同HCA压缩计算得到的C矩阵进行注意力计算，计算时也同样共享KV。

由此，则完成了高度压缩的HCA机制，HCA完整的架构也可以参考下图：



在实践中，先通过HCA来定位到上下文中关联度高的”大块信息“，随后在这些信息中进行CSA的稀疏注意力计算，实现“内容海选-内容精选-稀疏采样&精确计算”的三层处理，从而大幅优化超长上下文中的注意力计算量和KV-Cache消耗量。

3.其它注意力优化机制

同样，我们也看看业界在长上下文的注意力机制上还有哪些有意思的尝试。这里简要介绍Kimi、Qwen采用的混合线性注意力机制。注意力计算中Prefill的计算量和Decode的KV-Cache消耗量巨大，主要是由于 Q、K、V 和 Softmax 计算机制相互耦合所导致。线性注意力的关键技巧在于引入了一个核函数来打破 Softmax 的约束，使其可以基于矩阵运算的乘法结合律先算K*V，再作Q计算，如此则可以将计算复杂度由O(L^2)下降至O(L)，同时也不需要维护随序列长度增加的KV-Cache，仅需维护一个固定大小的隐状态矩阵，从而使计算量和显存消耗量均大幅降低。

Muon优化器：让训练更加稳定，快速收敛

举措一：在训练过程中将梯度正交化，使梯度在正交变换后各方向独立更新，减少震荡发散，提升稳定性的同时，训练收敛速度也更快。——这就好比你在用两只手配合拧一个魔方——左手拧一下，右手又拧回来，最后魔方越拧越乱。梯度正交化就像把两只手的动作"解耦"：左手只管左半边，右手只管右半边，互不牵扯，每拧一下都是真实的进步。

举措二：Logits爆炸会导致Softmax 分布极端化，和训练震荡等问题。DeepSeek V4将Q、K在计算注意力之前就先进行RMSNorm计算，即在softmax计算之前就规避掉QK^T矩阵中少数值过大的问题，也就从根本上解决了Logits爆炸的问题，从而提升训练的稳定性。——这就好比你在主持一个 10 人会议，规则是"每人发言加权平均后形成决策"。正常情况下大家音量相近，你的决策就能综合各方意见。但现在有一个人嗓子特别大，声音是其他 9 个人的 100 倍。最后加权平均的结果几乎完全由他一个人决定——其他 9 个人的意见完全被淹没。RMSNorm 就像给每人发

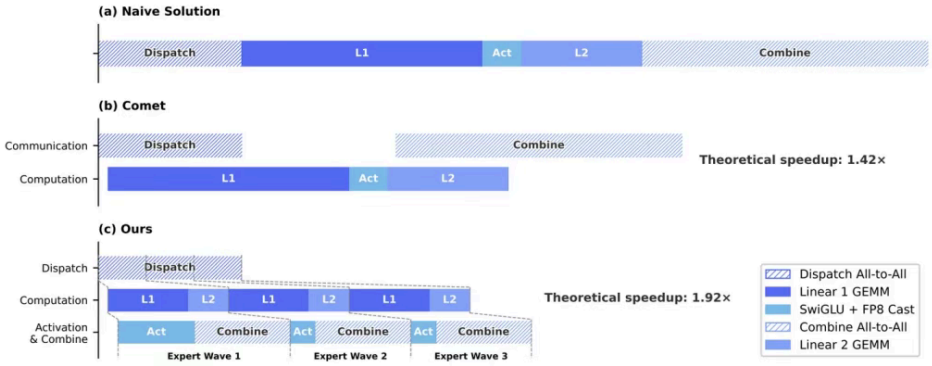
了一个定制的麦克风增益：无论你原本音量多大，话筒都自动调到合适的音量，确保每个人都能被公平地听见。

Infra层面的优化机制

前面聊到的三项核心优化机制瞄准的是"让模型变好"的问题，然而优雅的架构更加需要精细化的Infra方案，来"让硬件吃得饱"。接下来我们就来看DeepSeekV4针对Infra层优化的答案。

专家网络模块中更细粒度的计算通信重叠

MoE架构最初提出时，是按照“算完再通信，通信完继续算”的流程进行，这样带来的问题是在通信时算力模块闲置，在计算时通信模块闲置，硬件整体的资源利用率低。于是就有了计算通信重叠的优化思路，即在矩阵运算的同时，并行执行分发/回收的通信操作，这样可以适当提升硬件整体的利用率。但按阶段进行调度，很难让矩阵运算的阶段和通信阶段消耗的时间上是对齐的，也就是说依然存在气泡现象，DeepSeek V4提出了更加细粒度的调度方案，尽可能减少，甚至完全消除气泡的存在，从而大幅提升硬件的利用率。三种方案的执行时序对比详见下图：



这就好比我们同时使用洗衣机和烘干机时，由于洗衣和烘干的耗时往往不一致，如果中间不能打断，总会出现一台机器先完成、空转等另一台的情况。但如果两台机器都可以随时中断、交接——洗衣机刚洗完就把湿衣服送进烘干机、自己立刻洗下一批——整条流水线就能以最高效率运转。

另外，DeepSeek也洞察到，一款硬件计算能力C和通信带宽B的比值（C/B）是一个定值；而MoE模块计算时，一旦模型架构与量化方案确定，所需的计算量V_comp和通信数据量V_comm的比值（V_comp/V_comm）也将是一个定值。而MoE模块是算力密集型的，所以当Vcomp/Vcomm＞C/B时，理论上通信可以完全“隐藏”进计算，即实现气泡的完全消除。此外，部署MoE模块的硬件选型，也可以不必一味追求高带宽，够用即可。

使用TileLang来实现算子的高效开发

TileLang是一种面向tile的高级抽象语言，核心思想是数据流逻辑与调度策略解耦：前者用高层抽象语言简洁表达，后者通过少量标注交由编译器优化，达到以更简洁的代码表达复杂计算，并获得最优性能的效果。DeepSeek V4中包括如mHC、CSA、HCA等多种优化机制，如果纯靠CUDA编写算子将会非常复杂，通过TileLang编写，编译出的 kernel 性能对齐甚至超过手工 CUTLASS/cuBLAS 的水平。另外，TileLang的可移植性很好，同一套 TileLang 代码可以针对不

同后端编译，包括NVIDIA，也包括国产硬件。简单来说，TileLang就是让DeepSeek V4快速跑得起、跑得快、跑得远的关键支点。

批无关性&计算确定性

批无关性（Batch-Invariant）：

批无关是指在训练或推理过程中，无论batch如何切分，某个token落在哪个batch，它的输出结果都在 bit 级别上完全一致。形象比喻为：我和朋友一起去买奶茶，我喜欢精准50%糖的奶茶。无论店员做奶茶的顺序如何、或者我们一起下单的朋友有几个人，我的那杯都不能是51%糖、49%糖，必须是50%糖。——店员必须用同一套严格的固定工序做每一杯，不能因为订单多就开"批量快捷模式"走捷径，也不能因为做到后面累了就敷衍了事。

而具体的措施是，在注意力计算过程中，将“单条序列全程在单个 SM 内完成注意力计算，保证满载计算波的高吞吐”和“采用多 SM 协同处理单序列”两种kernel计算顺序完全对齐，使得同一条 query 无论走哪个kernel都能得到bit级别完全一致的结果，从而消除 batch 大小变化对输出的影响。另外，矩阵乘法这一侧的挑战更隐蔽，浮点加法不满足结合律，而 GEMM 内部本质上是大量浮点数的累加，累加顺序一变，bit 就漂。通用的GEMM库cuBLAS是动态调整乘法计算顺序和切分模式来追求性能最优的，同一个 GEMM 在不同 batch 下可能走不同路径，天然就是批相关的，所以需要使用DeepSeek开源的DeepGEMM矩阵乘法库来实现矩阵乘法的批无关。

计算确定性（Determinism）：

浮点加法不满足结合律，所以在大规模浮点累加过程中（注意力、矩阵乘法、梯度汇聚等），同一个数学上等价的计算可能因为执行顺序不同而产出不同的结果。我们还举买奶茶的例子，还是这一杯"50% 糖"的奶茶——按"糖浆 → 牛奶 → 冰块"的顺序调，和按"冰块 → 糖浆 → 牛奶"的顺序调，每样用料分量完全一样，但尝起来的糖度、奶感、冰感总会微妙地不一样。浮点加法就是这样一位挑剔的味觉——数学上明明等价的算式，换个加法顺序，最后一口总能尝出不同的味道。

为了保证浮点累加固定顺序，简单来说注意力反向传播、MoE 反向传播、mHC 的 split-k 归约——反向传播里这三个典型的"多对一"累加场景——V4 都强制要求累加顺序固定，"执行顺序"依然允许乱——只要最后把加法的括号结构钉死，整条训练就是 bit 级确定的。

FP4量化感知训练

核心思想是：与其让模型训练完再被粗暴量化、掉一波精度，不如在训练过程中就让模型"预演"低精度计算，提前适应量化带来的数值扰动。V4 在 post-training 阶段引入量化感知训练，目的是在部署时吃到推理加速 + 显存节省两重红利——但前提是：模型得先在训练中"习惯"低精度的世界。

但是量化也不是一股脑的一刀切，而是分模块量化：

- MoE 专家权重量化为FP4——MoE的参数量远大于ATTN模块，显存占用的大头在这里，压缩收益最大
- CSA闪电索引中的q、K也采用FP4量化，原因是这些事长上下文热点路径
- 但闪电索引的打分仅量化到BF16——这个分数是top-k排序用的，排序对数值精度敏感

这就好比你在搬家打包的时候，被子又大又重，直接抽真空压缩到极致，但衣服只能叠起来放，否则会压坏，不能正常穿了。

训练框架的优化

新优化器与旧分布式训练打架：V4 换上了新的优化器（Muon），它需要一次性拿到完整的参数梯度才能更新参数，但传统分布式训练会把参数切成小块分到各机器上，两者天然冲突——V4 重新设计了一套参数分组和分配方式，让 Muon 和分布式训练能共存，并把机器间传输梯度的通信量砍了一半。

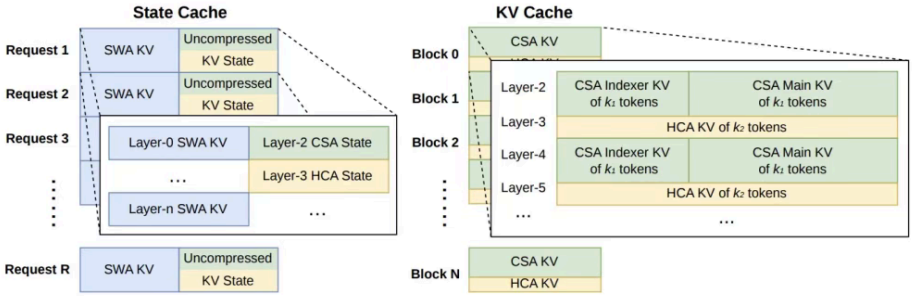
新残差结构拖慢流水线：V4 用的新残差结构（mHC）虽然效果更好，但会占更多显存、机器之间要传更多数据——V4 通过定制计算内核 + 选择性地"算过的不存、回头再算一遍" + 调整流水线节奏三招组合，把新残差带来的额外耗时压到只有 6.7%。

长文本训练时压缩段跨机器：训练超长文本时，一条文本会被切开分给不同机器处理，而 V4 的"KV 压缩"机制需要连续的一小段 token 一起压缩，这段有时正好跨在两台机器的边界上——V4 设计了一套两步走的机器间通信，让相邻机器先交换一下边界数据，再把压缩结果收拢重排，完美解决跨机器压缩的问题。

显存和算力的精细权衡：训练大模型时经常要在"多存一点中间结果省算力"和"少存一点省显存、用到了再算一次"之间做取舍——传统做法粒度太粗（要么整层都存、要么整层都重算），V4 做了一套精细到单个 tensor 的自动化机制，开发者只需要写一次前向逻辑 + 标注几个关键 tensor，框架就能自动帮你算出"重算哪些最省"。

推理框架的优化

KV Cache架构：由于V4 引入了包含CSA、HCA的混合注意力机制——它们的 KV 结构、大小、更新规则各不相同，导致每层的 KV Cache 长得都不一样。而传统的PagedAttention假设所有层 KV 形状一致，在 V4 面前直接失效，所以需要重新专门设计KV Cache架构，这个架构包括滑动窗口注意力的缓存和CSA/HCA混合注意力的KV Cache：



KV Cache持久化：想象一个公司的知识库，所有用户的system prompt、知识库内的文档，其实内容都是不经常变的，但在每次请求都要计算一次就会非常浪费资源。通用的做法是把算过的 KV 持久化到磁盘，下次命中直接读，跳过重算。而V4 的具体创新的点在于：面对自己的异构 KV（CSA/HCA 压缩 + SWA 未压缩 + 尾部 state），拆开分类存储，特别是给 SWA 提出了三档取舍方案，让这套机制在 V4 的混合架构下依然能用。

写在最后

写到这里，该停笔了。V4 没有发明新的轮子。mHC、CSA/HCA、Muon、TileLang、QAT——每一块拆开看原理，都似曾相识，但 V4 把它们逐一重新雕琢了一次，让它们都重新升华了，然而V4把它们组织在一起，便完成从架构到工程的一次系统级的闭环。V4 的动人之处，不只在每一块的精进，更在系统级的优雅。

当然这份技术报告我还有很多细节没有理解透彻，但可以肯定的是，它让开源又向前迈出了决定性的一步。

参考

DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence：https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf

DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models：https://arxiv.org/pdf/2512.02556

mHC: Manifold-Constrained Hyper-Connections：https://arxiv.org/pdf/2512.24880

ATTENTION RESIDUALS：https://arxiv.org/pdf/2603.15031

Kimi Linear: An Expressive, Efficient Attention Architecture：https://arxiv.org/pdf/2510.26692

Qwen3.5-Omni Technical Report：https://arxiv.org/pdf/2604.15804v2





阅读 1.8万

留言

写留言



还没有留言，来写首评